

ATTRIBUTE GRAMMAR FOR TYPE CHECKING FASE

AG = (G, A, R) where

A : expression.type , statement.returnType

G:

- (1) IntLiteral : expression $\rightarrow$ INT_CONSTANT
- (2) NumberLiteral : expression $\rightarrow$ NUMBER_CONSTANT
- (3) CharLiteral : expression $\rightarrow$ CHAR_CONSTANT
- (4) ArithmeticOperation : expression₁ $\rightarrow$ expression₂ expression₃
- (5) CompareOperation : expression₁ $\rightarrow$ expression₂ expression₃
- (6) LogicOperation : expression₁ $\rightarrow$ expression₂ expression₃
- (7) ArrayAccess : expression₁ $\rightarrow$ expression₂ expression₃
- (8) FieldAccess: expression₁ $\rightarrow$ expression₂ ID
- (9) Cast: expression₁ $\rightarrow$ expression₂ type
- (10) UnaryMinus: expression₁ $\rightarrow$ expression₂
- (11) UnaryNot: expression₁ $\rightarrow$ expression₂
- (12) Variable: expression $\rightarrow$ ID
- (13) Assignment: : statement $\rightarrow$ expression₁ expression₂
- (14) FunctionInvocation : expression₁ $\rightarrow$ expression₂ expression₃*
- (15) IfElseStatement : statement₁ $\rightarrow$ expression statement₂* statement₃*
- (16) InputStatement : statement $\rightarrow$ expression
- (17) LogStatement : statement $\rightarrow$ expression
- (18) ReturnStatement: statement $\rightarrow$ expression
- (19) WhileStatement: statement₁ $\rightarrow$ expression statement₂*
- (20) FuncDefinition : definition $\rightarrow$ type ID varDefinition* statement*

R:

- (1) expression.type = IntType.getInstance()
- (2) expression.type = NumberType.getInstance()
- (3) expression.type = CharType.getInstance()

- (4) `expression1.type = expression2.type.arithmetic(expression3.type)`
- (5) `expression1.type = expression2.type.comparison(expression3.type)`
- (6) `expression1.type = expression2.type.logic(expression3.type)`
- (7) `expression1.type = expression2.type.squareBrackets(expression3.type)`
- (8) `expression1.type = expression2.type.dot(ID)`
- (9) `expression1.type = expression2.type.canBeCastTo(type)`
- (10) `expression1.type = expression2.type.arithmetic()`
- (11) `expression1.type = expression2.type.logic()`
- (12) `expression.type = expression.definition.type`
- (13) `expression2.type.mustPromoteTo(expression1.type)`
- (14) `expression1.type = expression2.type.parenthesis(expression3*.stream().map(e -> e.type).toList())`
- (15) `expression.type.mustBeLogical(statement1)`
- (16) `expression.type.mustBeBuiltIn()`
- (17) `expression.type.mustBeBuiltIn()`
- (18) `expression.type.mustPromoteTo(statement.returnType)`
- (19) `expression.type.mustBeLogical(statement1)`
- (20) `statement*.forEach(stmnt -> stmnt.returnType = type.returnType)`